STOCKIQ · DOCUMENTAÇÃO DO FRONTEND

Frontend Aplicação Web

Documentação da interface do StockIQ: arquitetura, design system, mapa de telas, componentes e integração com a API.

Next.js 16 · App Router | React 19 | TypeScript | Tailwind CSS 4 | Recharts

Gabriel da Silveira Pessoni · Lívia Portela Ferreira
FATEC Franca — Desenvolvimento de Software Multiplataforma · 6º semestre

Projeto Integrador VI
Computação em Nuvem II · Mineração de Dados

Sumário

01	**Visão geral**	Propósito e números do front-end
02	**Stack tecnológica**	Frameworks e bibliotecas
03	**Arquitetura da aplicação**	App Router, grupos de rota e camadas
04	**Design system**	Tokens, tema claro/escuro e tipografia
05	**Layout e navegação**	AppShell, sidebar e topbar
06	**Componentes reutilizáveis**	Biblioteca de UI e componentes de domínio
07	**Mapa de telas**	Todas as rotas e seus objetivos
08	**Telas em destaque**	Produtos, inventário, IA analítica, scanner
09	**Autenticação e sessão**	Fluxo de login e proteção de rotas
10	**Integração com a API**	Cliente HTTP, tipos e consumo
11	**Aplicativo mobile**	Cliente complementar
12	**Execução e convenções**	Scripts, variáveis e padrões de código

01 Visão geral

O front-end do StockIQ é uma aplicação web em Next.js/React que dá forma à operação do armazém: cadastros, movimentações, lotes, inventário, alertas, relatórios e inteligência analítica. **Todas as telas operacionais leem e gravam dados reais na API**; não há mais dados de demonstração embutidos no código.

26	18	4	2
ROTAS	COMPONENTES UI	HOOKS	TEMAS

🎨

Design system

Tokens em CSS variables, tema claro/escuro e componentes sem regra de negócio.

🔌

Integrado à API

Cliente HTTP único, estados de carregamento/erro e renovação automática onde há processamento assíncrono.

👤

Perfis de acesso

Menu e ações respeitam o papel (admin, supervisor, operador, visualizador).

📷

Foto do produto

Envio para o S3 pela API, com pré-visualização, troca e remoção.

📋

Inventário assíncrono

Acompanha o ajuste de estoque feito pela fila e permite reprocessar.

🧠

IA analítica

Previsão de demanda, matriz ABC×XYZ e sugestões de compra vindas da API.

02 Stack tecnológica

Camada	Tecnologia	Finalidade
Framework	Next.js 16.2 (App Router)	Roteamento por pastas, layouts aninhados e renderização híbrida
UI	React 19	Componentização e hooks
Linguagem	TypeScript 5	Tipagem de domínio e das respostas da API
Estilização	Tailwind CSS 4 + CSS variables	Utilitários de estilo sobre os tokens do design system
Ícones	lucide-react	Conjunto único de ícones vetoriais
Gráficos	Recharts 3	Barras, linhas e áreas dos dashboards e da IA analítica
Fontes	Geist Sans · Geist Mono	Tipografia carregada via `next/font`
Qualidade	ESLint 9 (eslint-config-next) · `tsc`	Padronização e checagem estática

Camada	Tecnologia	Finalidade
Utilitários	clsx	Composição condicional de classes (helper `cn`)

03 Arquitetura da aplicação

As rotas públicas e autenticadas ficam em grupos de layout separados. O estado de sessão vem de um `AuthProvider` e o dos alertas de um `AlertsProvider`, ambos no layout autenticado.

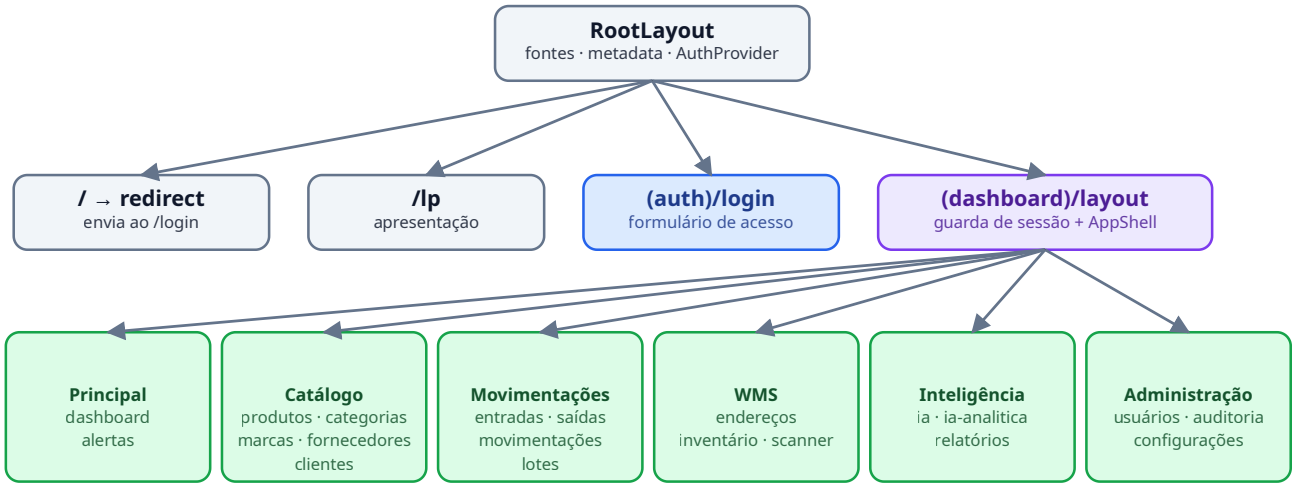


Figura 1 — Árvore de rotas e grupos de layout

Camadas

Camada	Papel
`app/`	Páginas e layouts (um diretório por rota)
`components/ui` · `components/layout`	Biblioteca de UI sem regra de negócio e estrutura de página (shell, sidebar, topbar)
`components/products` · `inventory` · `partners`	Componentes de domínio reutilizados por mais de uma tela
`hooks/` · `lib/`	Estado e utilidades; `lib/api.ts` é o único ponto de contato com o backend
`types/` · `constants/`	Tipagens de domínio e respostas da API; navegação e rótulos/cores de status

Estrutura de pastas

```
frontend/
├── app/
│   ├── layout.tsx · page.tsx · globals.css       # raiz, redirect e design tokens
│   ├── lp/                                       # página de apresentação
│   ├── (auth)/login/                             # autenticação
│   └── (dashboard)/dashboard/
│       ├── page.tsx                              # dashboard geral
│       ├── produtos/ (lista · novo · [id] · [id]/editar)
│       ├── categorias/ marcas/ fornecedores/ clientes/
│       ├── entradas/ saidas/ movimentacoes/ lotes/
│       ├── enderecos/ inventario/ scanner/
│       ├── ia/ ia-analitica/ relatorios/
│       └── usuarios/ auditoria/ alertas/ configuracoes/
├── components/  ui/ (18) · layout/ (3) · products/ · inventory/ · partners/
├── constants/   navigation.ts · status.ts
├── hooks/       useDebounce · useFetch · useSidebar · useTheme
├── lib/         api · auth-context · alerts-context · permissions · csv · fetch-all · image ·
utils · cn
└── types/       api.ts · movement.ts · user.ts
```

04 Design system

O visual é definido por tokens em `app/globals.css`, expostos como CSS variables e consumidos pelos componentes via Tailwind. O tema escuro redefine os mesmos tokens sob a classe `.dark`, sem duplicar componentes. O hook `useTheme` lê a preferência salva (ou a do sistema), aplica a classe no `<html>` e persiste a escolha.

Token	Claro	Escuro	Uso
`--brand`	#2563eb	#2563eb	Ações primárias, links, destaque
`--success`	#16a34a	#16a34a	Estoque ok, entradas, confirmações
`--warning`	#d97706	#d97706	Estoque baixo, vencimento próximo, pendências
`--danger`	#dc2626	#dc2626	Ruptura, perdas, exclusões, falhas
`--info`	#0891b2	#0891b2	Saídas, informações neutras, processamento
`--bg-base`	#ffffff	#1c1c1e	Superfície de cartões
`--bg-subtle`	#fafafa	#151517	Fundo da página
`--text-primary`	#1d1d1f	#f5f5f7	Texto principal
`--text-secondary`	#6e6e73	#a1a1a6	Texto de apoio
`--border`	#e9e9ec	#2c2c2e	Bordas e divisores

Grupo	Valores
Sidebar	224 px expandida · 64 px recolhida
Topbar	44 px de altura
Conteúdo	largura máxima de 1360 px
Raio	sm 6 · md 8 · lg 12 · xl 16 · 2xl 20 · full
Tipografia	Geist Sans (texto) e Geist Mono (códigos, SKUs)
Transição	200 ms · cubic-bezier(0.4, 0, 0.2, 1)

Domínio	Mapeamento semântico
Estoque	ok · Em Estoque · low · Estoque Baixo · critical · Crítico · out · Sem Estoque
Movimentação	Entrada · Saída · Transferência · Perda · Ajuste · Inventário
Produto	Ativo · Inativo · Descontinuado
Inventário	Planejado · Em Andamento · Em Revisão · Concluído
Ajuste de estoque	Na fila · Ajustando · Ajustado · Com pendências · Falhou

05 Layout e navegação

Todas as telas autenticadas compartilham o `AppShell`: sidebar agrupada por contexto de uso, topbar fina com busca, alertas (com contador de não lidos), tema e perfil, e uma área de conteúdo centralizada. A sidebar pode ser recolhida (64 px, só ícones) e, no mobile, abre como gaveta sobreposta (`useSidebar`).

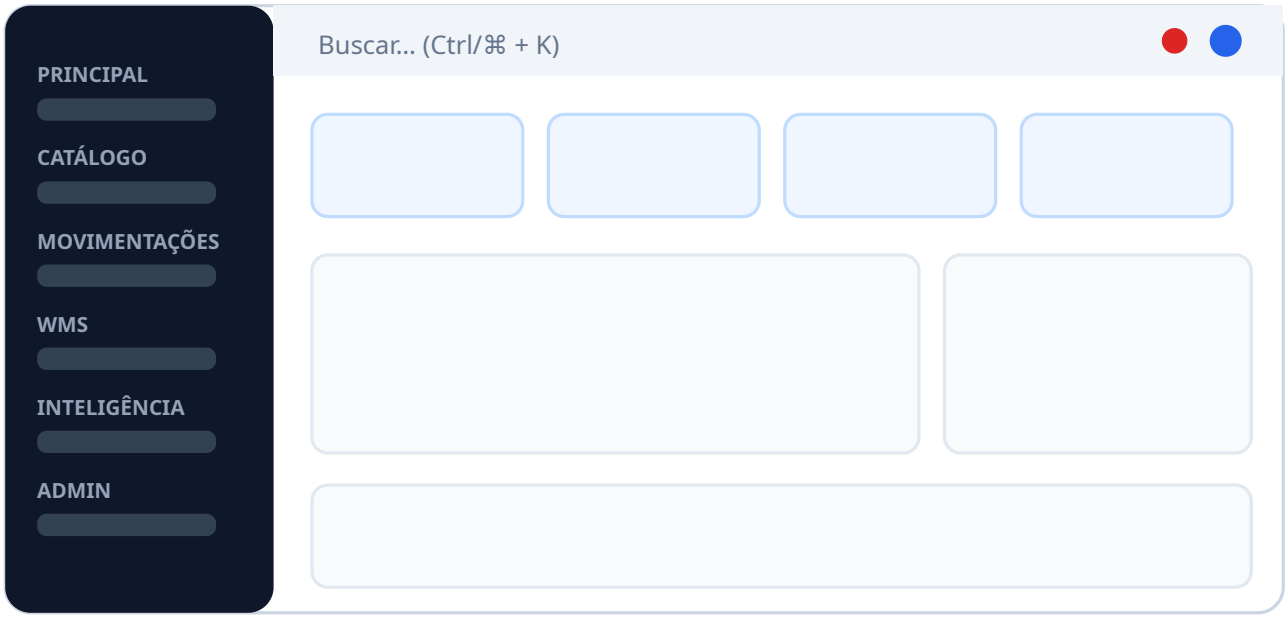


Figura 2 — Estrutura do AppShell (sidebar + topbar + conteúdo)

Grupo	Itens	Visível para
Principal	Dashboard · Alertas	Todos
Catálogo	Produtos · Categorias · Marcas · Fornecedores · Clientes	Todos
Movimentações	Entradas · Saídas · Histórico · Lotes	Todos
WMS	Endereços · Inventário · Scanner	Todos
Inteligência	IA — Produtos · IA Analítica · Relatórios	Todos
Administração	Usuários · Auditoria · Configurações	Usuários e Auditoria: admin e supervisor; Configurações: todos

Além do menu, botões de criação/edição/exclusão aparecem conforme o papel (`lib/permissions.ts`: `hasRole`, `STAFF`, `WRITERS`, `ADMIN`), espelhando as listas de `authorize(...)` do backend. A autorização definitiva continua sendo a da API.

06 Componentes reutilizáveis

Componente	Descrição	Variações e recursos
Button	Botão de ação	primary · secondary · ghost · danger · outline · tamanhos xs–lg · `loading` · ícones
Badge	Etiqueta de status	success, warning, danger, info, brand, default, secondary · ponto indicador
Card	Contêiner	CardHeader · CardTitle com descrição · preenchimento opcional
StatCard	Indicador numérico	Valor, rótulo, ícone, tendência
DataTable · Pagination	Tabela e paginação	Colunas configuráveis, ordenação, linhas clicáveis, total de itens
Input · Select · Textarea	Campos de formulário	Rótulo, ícones, erro e ajuda
Modal · ConfirmDialog	Diálogos	Título, descrição, rodapé de ações; confirmação de exclusão
Tabs · Breadcrumb	Navegação local	Troca de painéis; trilha de navegação
Alert · EmptyState · Loading · Skeleton	Estados da interface	Erro, lista vazia, carregamento e placeholders animados
Avatar	Identidade do usuário	Iniciais ou imagem

Componente de domínio	Função
`ProductForm`	Cadastro/edição de produto em abas (informações, estoque e preço, logística), compartilhado por `/produtos/novo` e `/produtos/[id]/editar`
`ProductImageUploader` · `ProductImage`	Escolher/arrastar a foto, pré-visualizar, trocar e remover; exibição com ícone de reserva quando não há foto ou o link expirou
`InventoryJobBanner`	Situação do ajuste de estoque do inventário (na fila, ajustando com progresso, ajustado, pendências, falhou) e botão **Reprocessar**
`PartnerManager`	CRUD compartilhado de fornecedores e clientes
`AppShell` · `Sidebar` · `Topbar`	Estrutura das telas autenticadas

Hook / utilitário	Função
`useFetch`	Carrega um endpoint, controla `loading` / `error`, ignora respostas atrasadas e expõe `reload`
`useDebounce`	Adia a propagação de um valor (buscas)
`useSidebar` · `useTheme`	Sidebar recolhida/gaveta mobile · tema claro/escuro persistido
`useAuth` · `useAlerts`	Usuário, login/logout · alertas, contador de não lidos e marcação de leitura (atualiza a cada 60 s)
`fetchAll`	Percorre todas as páginas de um endpoint paginado (limite de 100 por página)
`downloadCsv`	Exporta linhas em CSV UTF-8 com BOM (Excel) — usado no pedido sugerido da IA Analítica
`validateImage`	Regras de foto no cliente (tipo e 5 MB), espelhando o backend
`cn` · `formatCurrency` · `formatNumber` · `formatDate`	Classes condicionais e formatação pt-BR

07 Mapa de telas

O sistema reúne **26 rotas**. Todas as telas operacionais usam dados reais da API; as exceções são a página de apresentação (`/lp`, conteúdo estático) e `IA — Produtos` (protótipo: aceita e pré-visualiza a imagem, mas o modelo de visão ainda não está integrado e a tela **não** inventa resultados).

ACESSO

Rota	Descrição	Dados
`/`	Redireciona para o login	Estática
`/lp`	Página de apresentação do produto	Estática
`/login`	Acesso com e-mail e senha	API real

VISÃO GERAL

Rota	Descrição	Dados
`/dashboard`	Indicadores, tendência mensal, distribuição por categoria, curva ABC, movimentações recentes	API real
`/dashboard/alertas`	Alertas de estoque e validade por severidade, com leitura por usuário	API real

CATÁLOGO

Rota	Descrição	Dados
`/dashboard/produtos`	Lista em tabela ou cartões, busca, filtros, foto e exclusão	API real
`/dashboard/produtos/novo`	Cadastro com categoria, marca, fornecedor, preços, limites e foto	API real
`/dashboard/produtos/[id]`	Detalhe completo, foto, lotes, endereços e últimas movimentações	API real
`/dashboard/produtos/[id]/editar`	Edição do produto e troca/remoção da foto	API real
`/dashboard/categorias · marcas`	CRUD de categorias (com cor) e marcas	API real
`/dashboard/fornecedores · clientes`	Cadastro de fornecedores e clientes	API real

MOVIMENTAÇÕES

Rota	Descrição	Dados
`/dashboard/entradas · saidas`	Registro de entradas e saídas de estoque	API real
`/dashboard/movimentacoes`	Histórico geral com filtros	API real
`/dashboard/lotes`	Controle de validade de lotes	API real

WMS

Rota	Descrição	Dados
`/dashboard/enderecos`	Endereçamento físico e ocupação	API real
`/dashboard/inventario`	Contagens completa/parcial/cíclica, divergências e acompanhamento do ajuste de estoque	API real
`/dashboard/scanner`	Leitura/entrada de código para localizar produtos (modos Entrada, Saída, Consulta e Transferência)	API real

INTELIGÊNCIA

Rota	Descrição	Dados
`/dashboard/ia`	Identificação de produto por imagem (modelo ainda não integrado)	Protótipo
`/dashboard/ia-analitica`	Previsão de demanda, matriz ABC×XYZ, sugestões de compra e exportação do pedido	API real
`/dashboard/relatorios`	Relatórios gerenciais (estoque, movimentos, lotes, ocupação, ABC, inventário, fornecedores)	API real

ADMINISTRAÇÃO

Rota	Descrição	Dados
`/dashboard/usuarios`	Gestão de usuários e perfis	API real
`/dashboard/auditoria`	Trilha de operações críticas	API real
`/dashboard/configuracoes`	Perfil, segurança, aparência, empresa, preferências e sistema	API real

08 Telas em destaque

Produtos e foto

A lista alterna entre tabela e cartões e mostra a foto de cada produto. No cadastro/edição, o card **Foto do Produto** aceita arrastar ou escolher um arquivo (JPEG, PNG, WebP ou GIF até 5 MB), mostra a pré-visualização e permite trocar ou remover. A foto só é enviada ao salvar: o formulário cria/atualiza o produto e então chama `POST /products/:id/image` (ou `DELETE` para remover). Se o produto foi salvo mas o envio da foto falhou, a tela avisa e, ao salvar de novo, tenta apenas a foto, sem duplicar o produto.

Inventário e ajuste de estoque assíncrono

Ao **Concluir** um inventário, a API enfileira o ajuste de estoque e a tela exibe um banner com o andamento: *Na fila*, *Ajustando o estoque… X de Y* (com barra de progresso), *Estoque ajustado*, *Ajuste concluído com pendências* ou *Falhou*. Enquanto houver job em andamento, a lista e os itens abertos são atualizados sozinhos a cada 3 segundos. Admin e supervisor veem o botão **Reprocessar** quando o job falhou ou terminou com pendências; na tabela de itens, cada divergência mostra *Ajustado*, *Aguardando…* ou o motivo da pendência manual.

IA Analítica



Previsão de demanda

Gráfico de real × previsto com faixa de incerteza e a origem do modelo (ML ou média móvel).



Matriz ABC × XYZ

Valor movimentado × regularidade da demanda, com giro e % do valor.



Insights e compras

Alertas acionáveis e sugestões de compra com urgência, motivo, custo estimado e exportação CSV do pedido.

A tela indica a origem da previsão (modelo de ML quando disponível, senão **média móvel**), as métricas do último teste quando há modelo e o prazo de reposição configurado.

Scanner e configurações

O scanner tem quatro modos (Entrada, Saída, Consulta e Transferência): o código lido localiza o produto e exibe estoque e dados, com histórico das leituras recentes. Em Configurações, as abas cobrem perfil, segurança (troca de senha), aparência (tema), empresa, preferências (dias de alerta de validade e prazo de reposição) e informações do sistema; a edição de empresa e preferências é restrita ao administrador.

09 Autenticação e sessão

O `AuthProvider` envolve a aplicação, mantém o usuário logado e expõe `login` e `logout` pelo hook `useAuth`. O token JWT é guardado no `localStorage` (chave `wms_token`) e enviado em cada requisição.

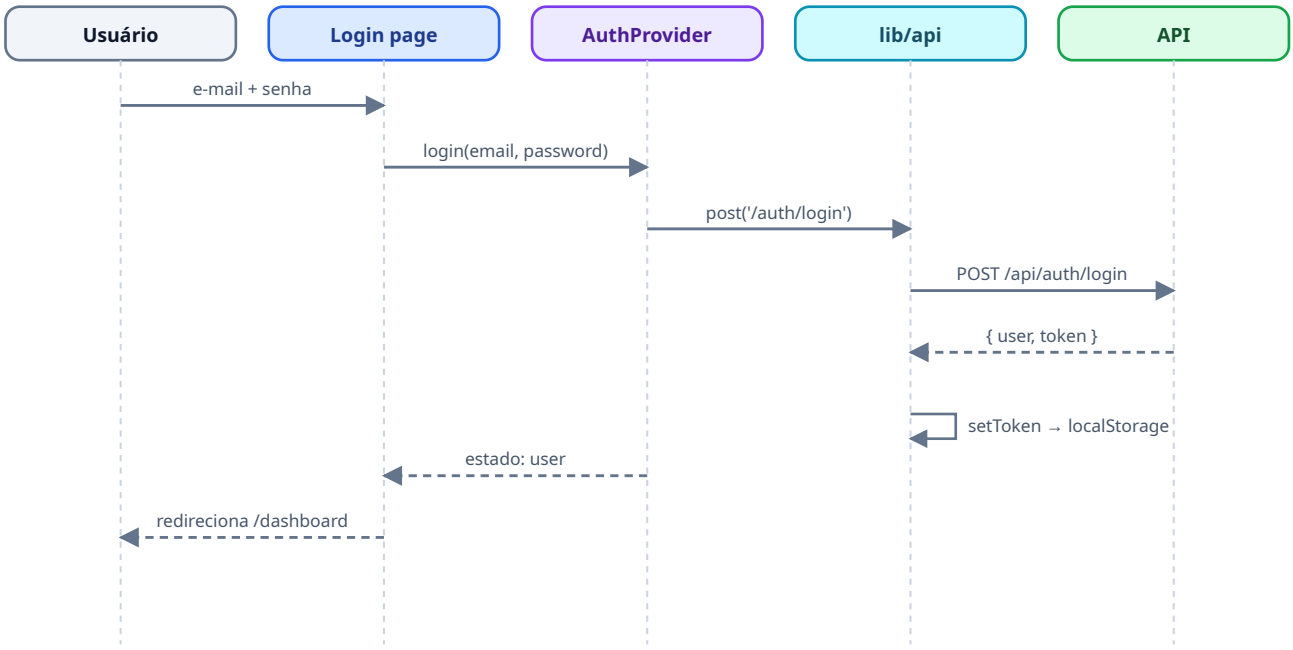


Figura 3 — Fluxo de login

O layout `(dashboard)/layout.tsx` aguarda a verificação da sessão (`GET /auth/profile`) e, sem usuário, redireciona para `/login`. Qualquer resposta **401** da API limpa o token e leva ao login. O perfil logado

aparece na sidebar, com a ação de sair.

10 Integração com a API

`lib/api.ts` centraliza a comunicação: monta a URL base (`NEXT_PUBLIC_API_URL`, padrão `http://localhost:3001/api`), injeta o `Authorization`, serializa o corpo, constrói a query string e converte erros em `ApiError`. Para **upload**, `api.upload(path, FormData)` envia multipart sem fixar o `Content-Type` (o navegador define o *boundary*).

```
api.get<T>(path, params?)          // GET com query string
api.post<T>(path, body?)           // POST JSON
api.patch<T>(path, body?)          // PATCH JSON
api.put<T>(path, body?)            // PUT JSON
api.delete<T>(path)                // DELETE
api.upload<T>(path, formData)      // POST multipart (fotos)
```

Tela	Operações
Login / sessão	`POST /auth/login` · `GET /auth/profile`
Dashboard	`GET /dashboard` e `/dashboard/*` (tendência, categorias, top produtos, ABC) · `GET /reports/inventory` · `/reports/lots`
Produtos	`GET/POST/PATCH/DELETE /products` · `POST/DELETE /products/:id/image` · `GET /categories` · `/brands` · `/suppliers`
Entradas · Saídas · Histórico	`POST /movements` · `GET /movements` · `GET /products` · `/suppliers` · `/warehouse`
Lotes · Endereços	`/lots` · `/warehouse` · `/warehouse/stats`
Inventário	`/inventory` · `POST /inventory/:id/items/:itemId/count` · `POST /inventory/:id/reprocess`
Alertas	`GET /alerts` · `PATCH /alerts/read-all` · `PATCH /alerts/:id/read`
IA Analítica · Relatórios	`GET /analytics` · `GET /reports/*`
Usuários · Auditoria · Configurações	`/users` · `GET /audit` · `/settings` · `PATCH /auth/profile` · `PATCH /auth/password`

As respostas são tipadas na borda (`types/api.ts`). Listagens paginadas chegam como `{ data, total, page, limit, totalPages }`; categorias e marcas, como arrays simples. Imagens de produto chegam como **URL assinada temporária**; se expirar, o componente `ProductImage` volta ao ícone de reserva.

11 Aplicativo mobile

O repositório inclui também um aplicativo mobile (Expo / React Native, `expo-router`) que consome a **mesma API**, com telas equivalentes às principais do web (produtos, movimentações, inventário, scanner,

lotes, endereços, relatórios, IA analítica, usuários e configurações). Este documento descreve a aplicação web.

12 Execução e convenções

```
cd frontend
npm install
npm run dev        # http://localhost:3000  (backend em http://localhost:3001)
npm run build      # build de produção
npm run start      # serve o build
npm run lint       # ESLint
```

Item	Convenção
`NEXT_PUBLIC_API_URL`	URL base da API (padrão `http://localhost:3001/api`)
Dados	Sempre pela `lib/api` (ou `useFetch`); nada de `fetch` espalhado pelas telas
Estilo	Somente tokens (`var(--…)`) e utilitários Tailwind; sem cores fixas nas telas
Verificação	`tsc --noEmit`, `eslint` e `next build` sem erros na última verificação (19/10/2026); a interface não foi exercitada em navegador por teste automatizado